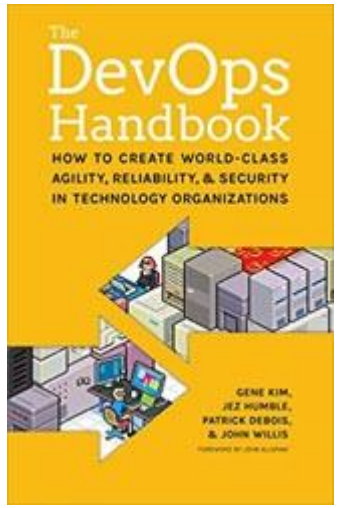
Жизнен цикъл на софтуерна разработка

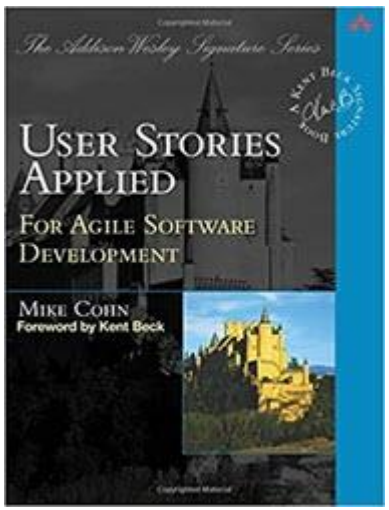
Литература

1. Буч Г., Обектно-оринтированое проектирование с примерами применение:пер.с анг. – М.,Конкорд, 1992. – 519 с.
2. Зеленков Г., Разработване и изучаване на пакети приложни програми, Варна, 1989
3. Н.Манева, А. Ескенази, Софтуерни технологии, АНУБИС, С., 2001
4. Илиева С., Лилов В., Манова И., Изграждане на софтуерни приложения, Унив. Изд. „Св. Климент Охридски”, София, 2006

Robert C. Martin Series
Clean Code
A Handbook of Agile Software Craftsmanship
Robert C. Martin


The
DevOps
Handbook
HOW TO CREATE WORLD-CLASS
AGILITY, RELIABILITY, & SECURITY
IN TECHNOLOGY ORGANIZATIONS
GENE KIM,
JEZ HUMBLE,
PATRICK DEBOIS,
& JOHN WILLIS

CODE
COMPLETE
2


The Addison Wesley Signature Series
User Stories
Applied
For Agile Software
Development
Mike Cohn
Foreword by Kent Beck

O'REILLY
Head First
Design Patterns
A Brain-Friendly Guide
Eric Freeman & Elisabeth Robson

Design Patterns
Elements of Reusable
Object-Oriented Software
Erich Gamma
Richard Helm
Ralph Johnson
John Vlissides
Foreword by Grady Booch
ADDISON-WESLEY PROFESSIONAL COMPUTING SERIES

КЛАСИЧЕСКИ МЕТОДОЛОГИИ ЗА РАЗРАБОТА НА СОФТУЕР

Софтуерният процес

Структурирано множество от дейности, нужни за разработката на софтуерна система като:

- Спецификация;
- Проектиране;
- Валидация;
- Развитие;

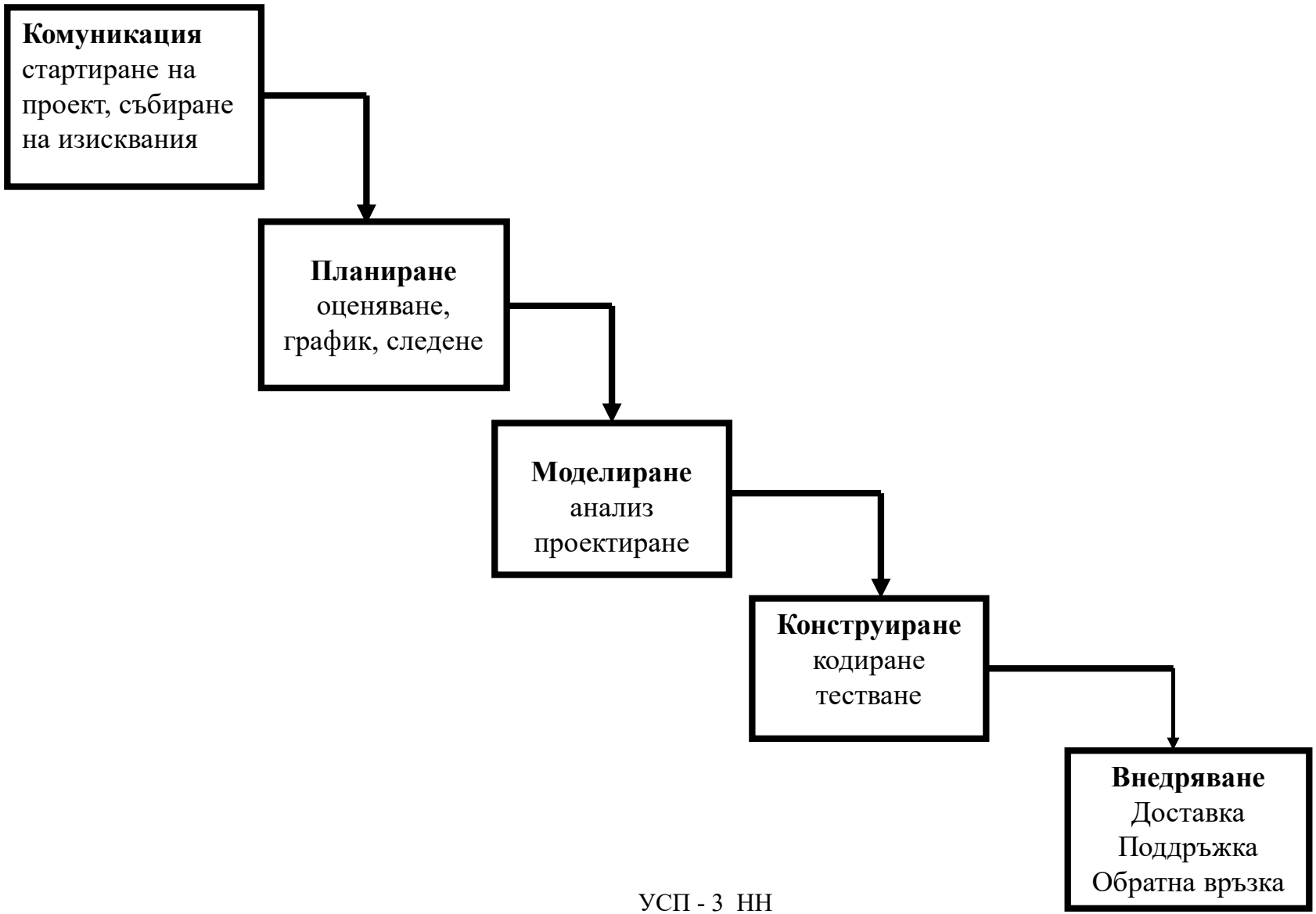
Моделът на софтуерния процес е абстрактно представяне на процеса. Той представлява описание на процеса от определена гледна точка.

КАСКАДЕН (ВОДОПАД)

Каскаден (водопад).

Моделът на водопада е най-старият метод на структурирано разработване на софтуер, предложен, от Ройс през 1970 г. Той осигурява теоретичната база за останалите модели на процеси и все още широко се използва в практиката.

Каскаден (водопад)



Характеристики на модела

- ясно разграничен и разбираем процес;
- всяка стъпка в модела завършва със създаване на документи - например документ с изискванията, документ с дизайна, програмен код, документи за тестването;
- всяка дейност трябва да бъде напълно завършена, преди да се премине към следваща, а това става, като се одобрят документите;
- ясно са дефинирани входовете и изходите на дейностите, както и интерфейсите между отделните стъпки;
- ясно са дефинирани ролите на разработчиците на софтуер.

Проблемите са:

- Реалните проекти рядко следват последователния поток на модела
- Трудно е за потребителите да поставят своите изисквания за софтуера наведнъж – което е изискване за този тип модели
- Потребителите трябва да са търпеливи – работеща версия на системата ще бъде налична не по-рано от крайния срок за изпълнение на проекта.
- Разделянето на проекта на етапи не е гъвкаво
- Трудно е да се реагира на променящите се изисквания на потребителите

Приложимост на модела

Като се имат предвид споменатите проблеми, използването на този модел е подходящо, когато изискванията са осъзнати и ясно формулирани в началото и когато проектите са ясно организирани в смисъл на ясно дефинирани роли. Други случаи на подходящо използване са при повторяеми проекти и/или големи проекти, за които времето и бюджетът не са критични.

Фази на Waterfall модела

- Анализ и дефиниране на изискванията
- Проектиране на системата и софтуера
- Разработка и тестване на елементите
- Интегриране и тестване на системата
- Експлоатация и поддръжка

Основния недостатък на waterfall модела са затрудненията за приспособяване към промените след като процесът е започнал. Преди да се започне следващата фаза, предишната трябва напълно да е завършена.

Проблеми при Waterfall модела

- Твърдото разделяне на проекта на отделни етапи прави трудни промените, дължащи на променящи те се изисквания на клиента.
- Следователно, този модел е подходящ, само когато изискванията са установени и промените са твърде ограничени по време на проектирането.
- Малко бизнес системи имат установени изисквания
- Waterfall моделът се използва най-вече за проектирането на големи системи, когато системите се разработват на различни места.

ЕВОЛЮЦИОННА РАЗРАБОТКА

Еволюционна разработка

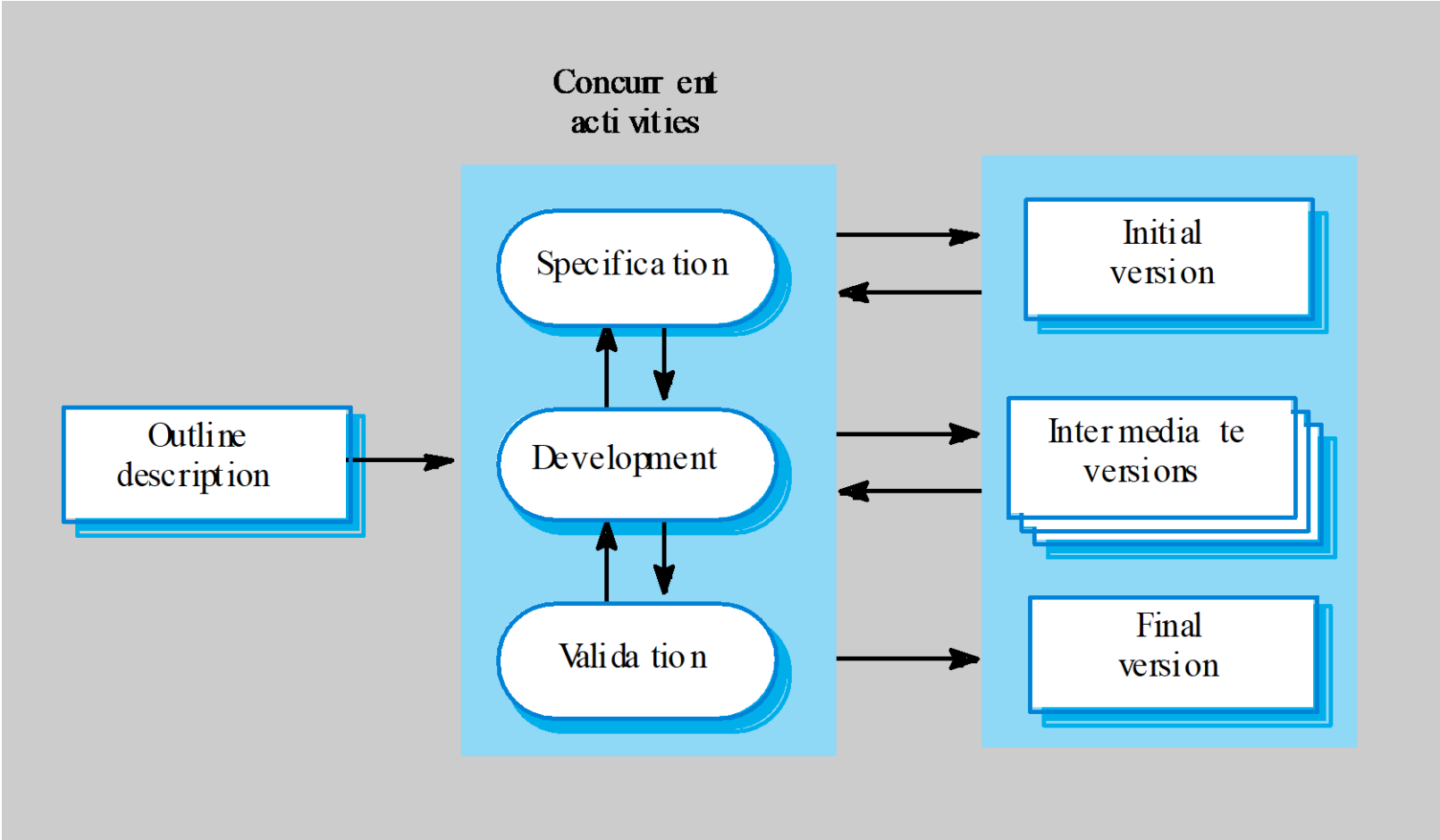
- Постепенна разработка

  Целта е да се работи с клиента и да се създаде завършена система от схематична начална спецификация. Трябва да започне с добре разбрани изисквания и да се добавят нови функции по предложение на клиента

- Прототипиране чрез изхвърляне

  Целта е да се разберат системните изисквания. Започва се със недобре изяснени изисквания

Еволюционна разработка...



Еволюционна разработка...

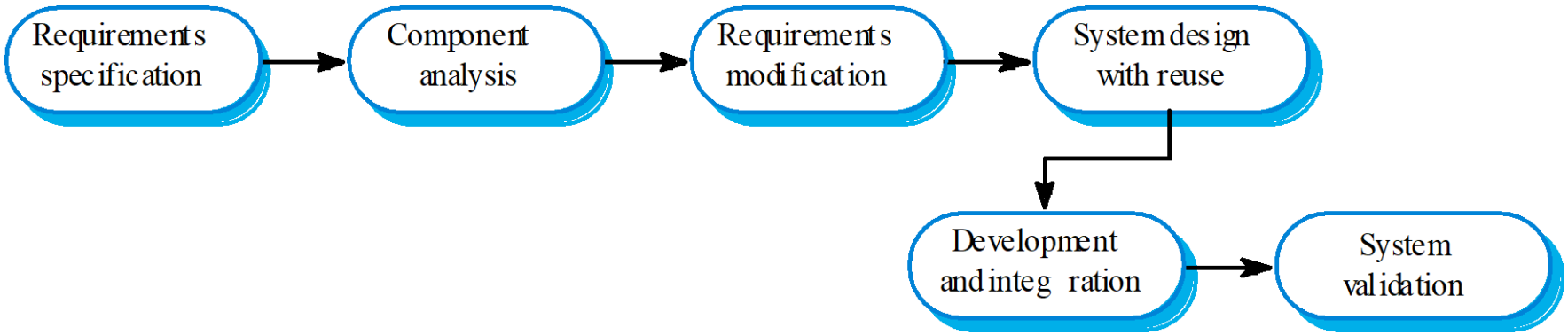
- Проблеми
  - Липса на прозрачност
  - Често системата е зле структурирана
  - Може да се изискват специални умения (напр. езици за бързо създаване на прототипи)
- Приложение
  - За малки или средно големи интерактивни системи
  - За части от големи системи (напр. потребителски интерфейс)
  - За системи с къс жизнен цикъл

СОФТУЕРЕН ИНЖЕНЕРИНГ БАЗИРАН НА КОМПОНЕНТИ

Софтуерен инженеринг базиран на компоненти

- Базира се на многократно използване, като системите се сглобяват от съществуващи компоненти или COTS (Commercial-off-the-shelf) системи
- Етапи на процеса:
  - Анализ на компонентите;
  - Модификация на изискванията;
  - Проектиране на системата с многократно използване;
  - Разработка и интеграция
- Този подход все повече се използва с появата на стандарти за компонентите.

Разработка с многократно използване



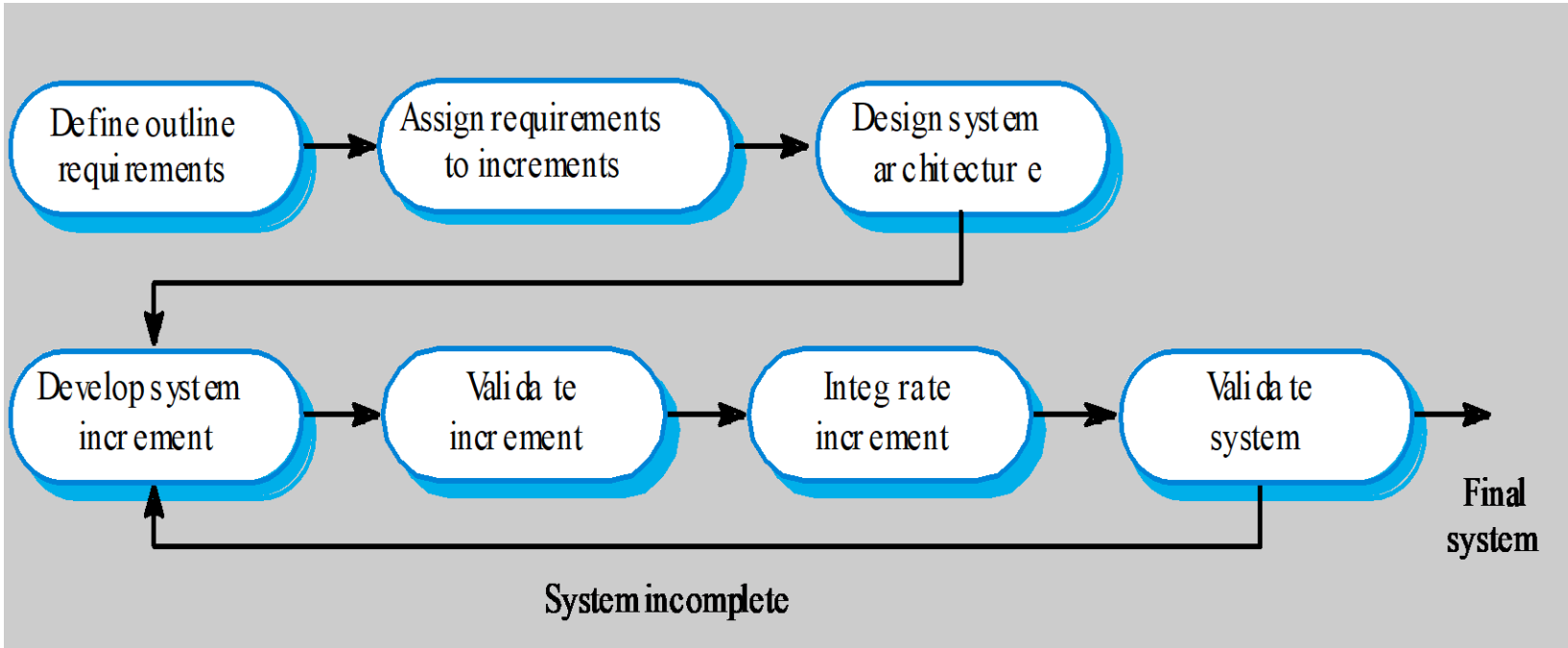
Итерации в процеса

- Изискванията към системата ВИНАГИ се определят по време на процеса, така че винаги итерациите, в които се преработват по-ранни етапи, са винаги част от процеса
- Итерациите могат да се приложат в кой да е модел.
- Два (свързани помежду си) подхода:
  - Доставка на стъпки (increments);
  - Спирална разработка.

Доставка на стъпки

- Вместо да се доставя системата наведнъж, разработката и доставката са разбити на стъпки, като на всяка стъпка се доставя част от системата и от изискваната функционалност.
- На потребителските изисквания се дават приоритети и изискванията с най-високи приоритети се включват в първите стъпки.
- Веднъж започнала разработката на една стъпка, съответните изисквания се замразяват, макар че изискванията за следващите стъпки могат да се променят.

Разработка на стъпки



Предимства на разработката на стъпки

- Със всяка стъпка се доставя нещо работещо, така че функционалността на системата е по-рано достъпна.
- Ранните стъпки служат за прототип, за да подпомогне извличането на изискванията за следващите стъпки
- По-малък риск за целия проект.
- Услугите с по-висок приоритет биха били по-тествани

ЕКСТРЕМНО ПРОГРАМИРАНЕ

Екстремно програмиране

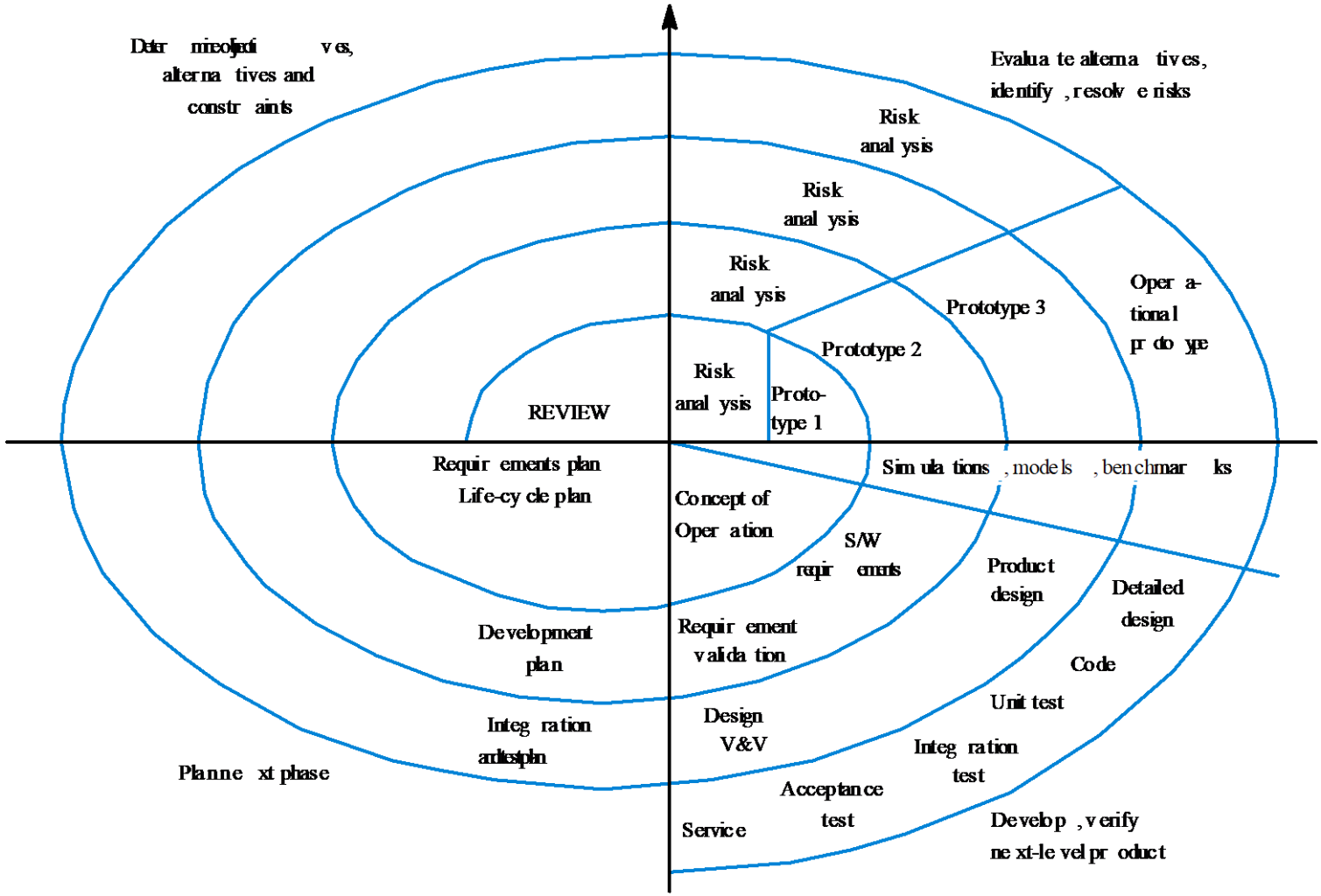
- Подход базиран на разработката и доставката на много малки стъпки.
- Разчита на постоянно подобряване на кода, въвличане на потребителя в тима и програмиране по двойки.
- Един от методите за бързо програмиране

СПИРАЛОВИДЕН МОДЕЛ

Разработка по спирала

- Процесът е представен като спирала, а не като последователност от дейности с връщане назад.
- Всяка витка на спиралата представлява фаза в процеса
- Няма фиксирани фази като спецификация или проектиране – витките в спиралата се избират в зависимост от това какво се изисква.
- Рискът се оценява явно и решава по време на процеса.

Спирален модел на процеса



Сектори в спиралния модел

- Поставяне на цели
  - Идентифицират се специфичните за фазата цели
- Оценка на риска намаляването му
  - Оценява се риска и се планират дейностите за намаляването му.
- Разработка и валидиране
  - Избира се модел за разработката на системата, който може да е всеки от общите модели
- Планиране
  - Проектът се преглежда и се планира следващата фаза на спиралата

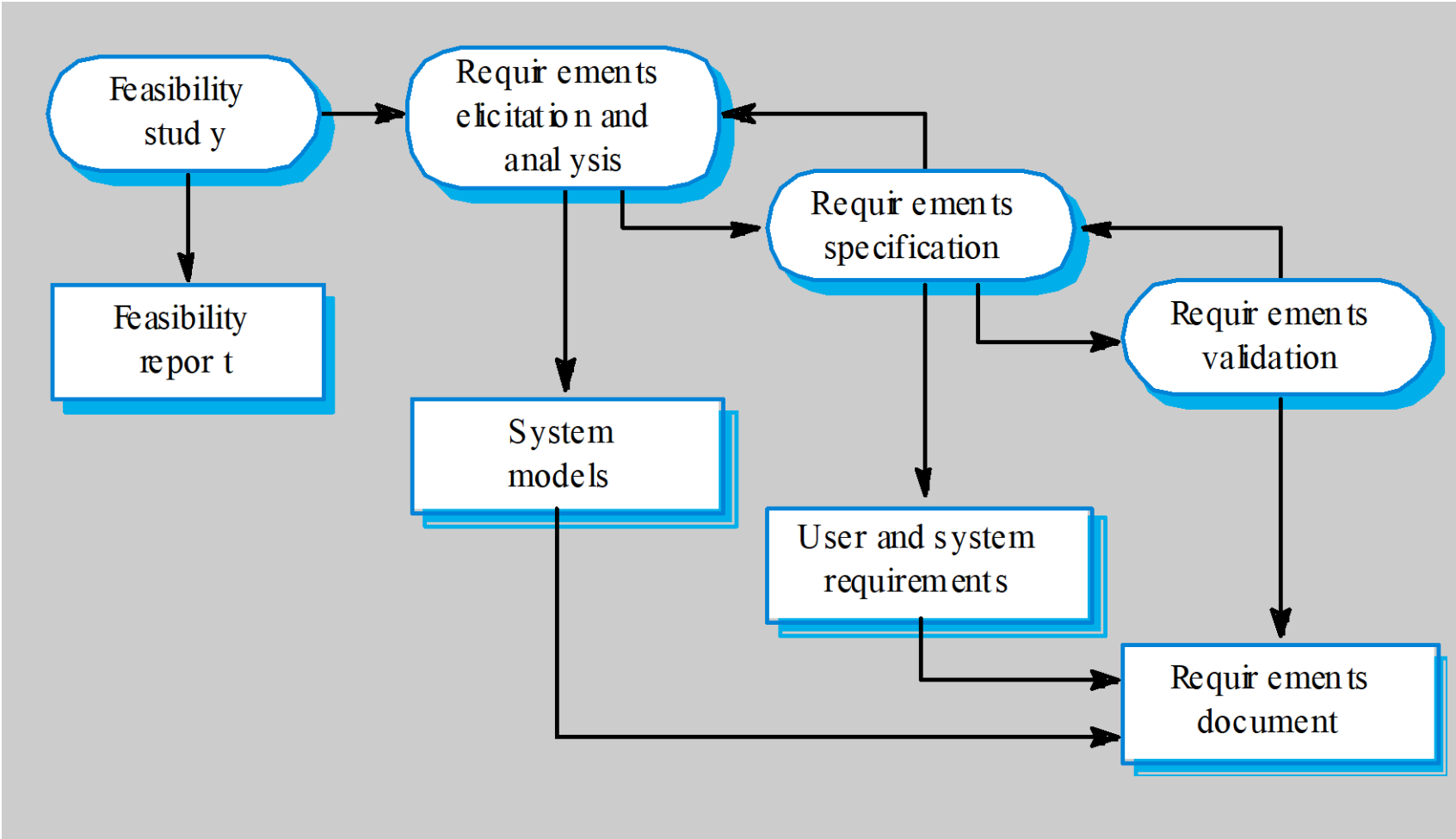
Дейности на процеса

- Спецификация на софтуера
- Проектиране на софтуера и осъществяване
- Валидиране
- Еволюция

Спецификация на софтуера

- Процесът на установяване на какви услуги се изискват и на ограниченията върху разработката и експлоатацията на системата
- Инженеринг на изискванията
  - Изучаване на възможностите за реализация – да продължим или да спрем;
  - Извличане на изискванията и анализът им;
  - Спецификация на изискванията;
  - Валидиране на изискванията;

Инженеринг на изискванията



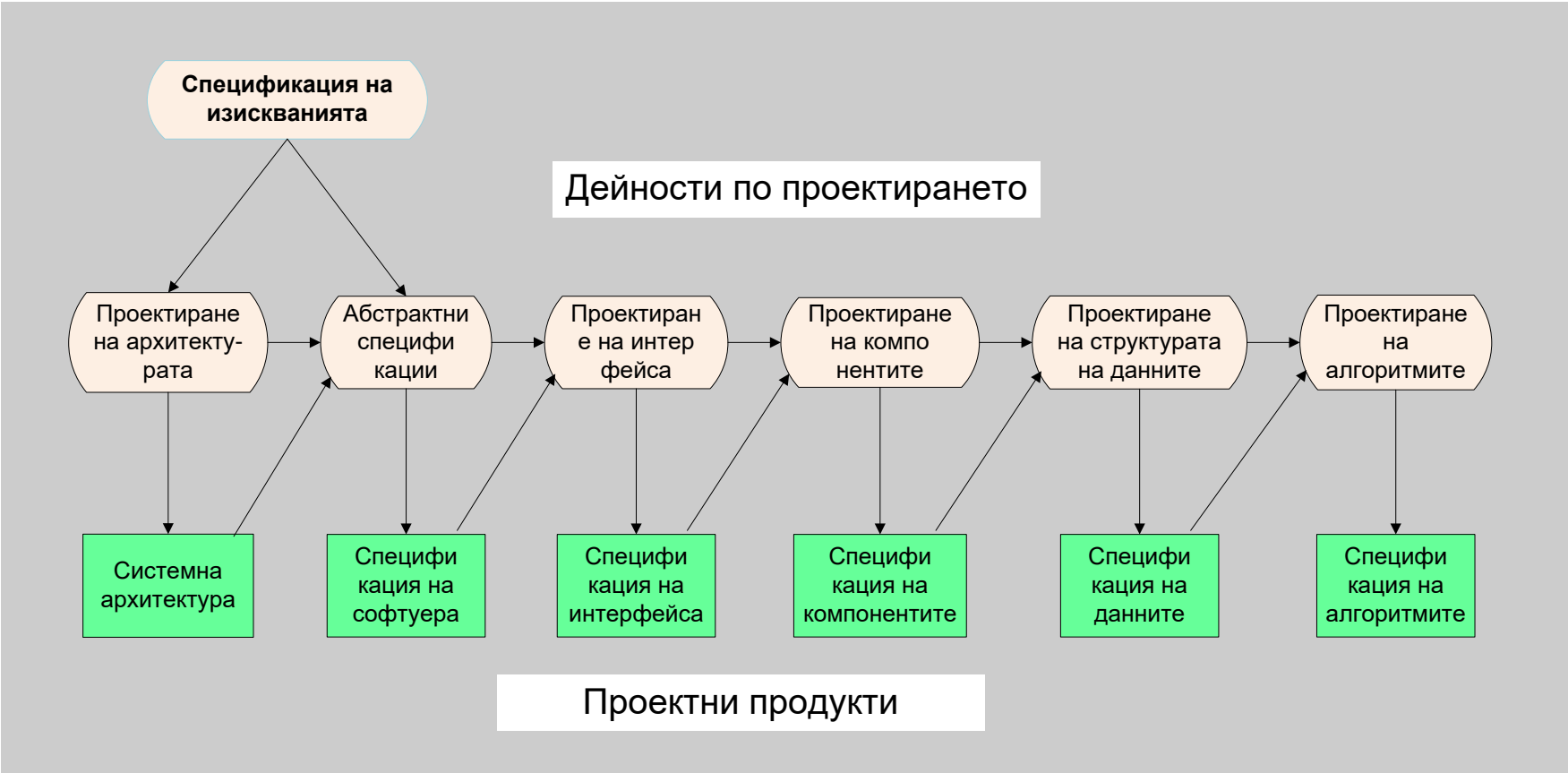
Проектиране и осъществяване

- Процесът на преобразуване на спецификацията в изпълнима система
- Проектиране
  - Проектиране на структурата на софтуера, която реализира спецификацията.
- Осъществяване
  - Превежда тази структура в изпълнима програма
- Тези две дейности са тясно свързани и могат да се припокриват

Дейности на проектирането

- Архитектурен дизайн
- Абстрактна спецификация
- Дизайн на интерфейса
- Проектиране на компонентите
- Проектиране на структурата на данните
- Проектиране на алгоритмите

Процес на проектиране на софтуера



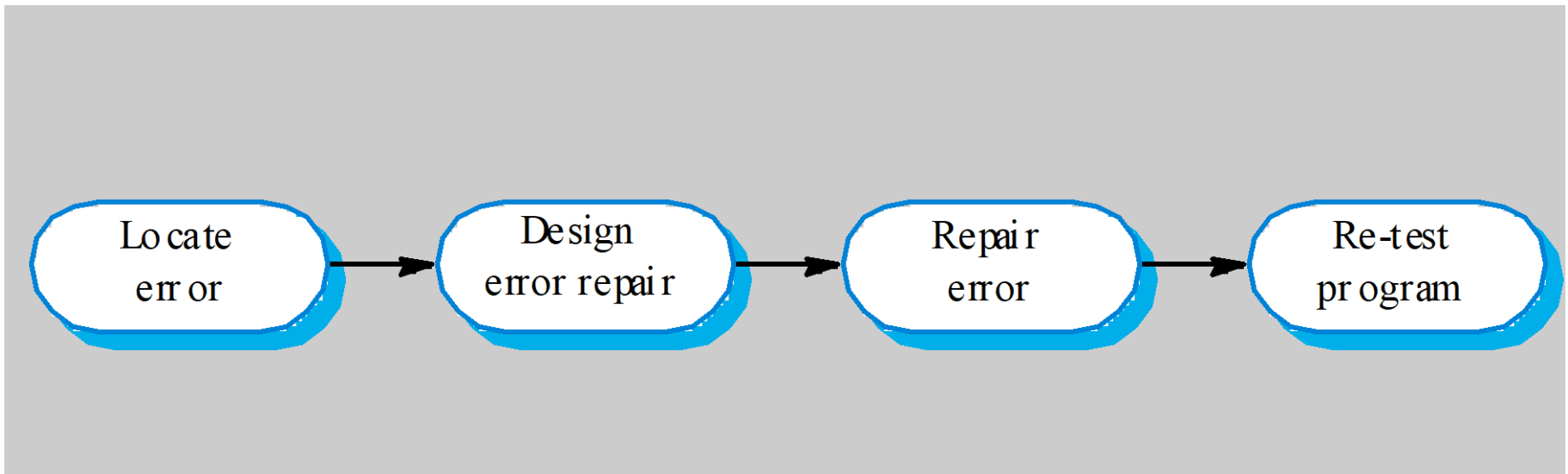
Структурни методи

- Систематични към разработката на софтуерния проект.
- Проектът обикновено се документира като съвкупност от графични модели.
- Възможни модели:
  - Обектен модел;
  - Модел на последователностите (sequence);
  - Модел на състоянията (state transition);
  - Структурен модел;
  - Поток на данните

Програмиране и настройка (**debugging**)

- Превод на проекта в програма и отстраняване на грешките от нея.
- Програмирането е лична дейност и не съществува обобщен процес на програмиране.
- Програмистите осъществяват програма, тестват я, за да открият дефекти и отстраняват тези дефекти в процеса на настройка (debugging).

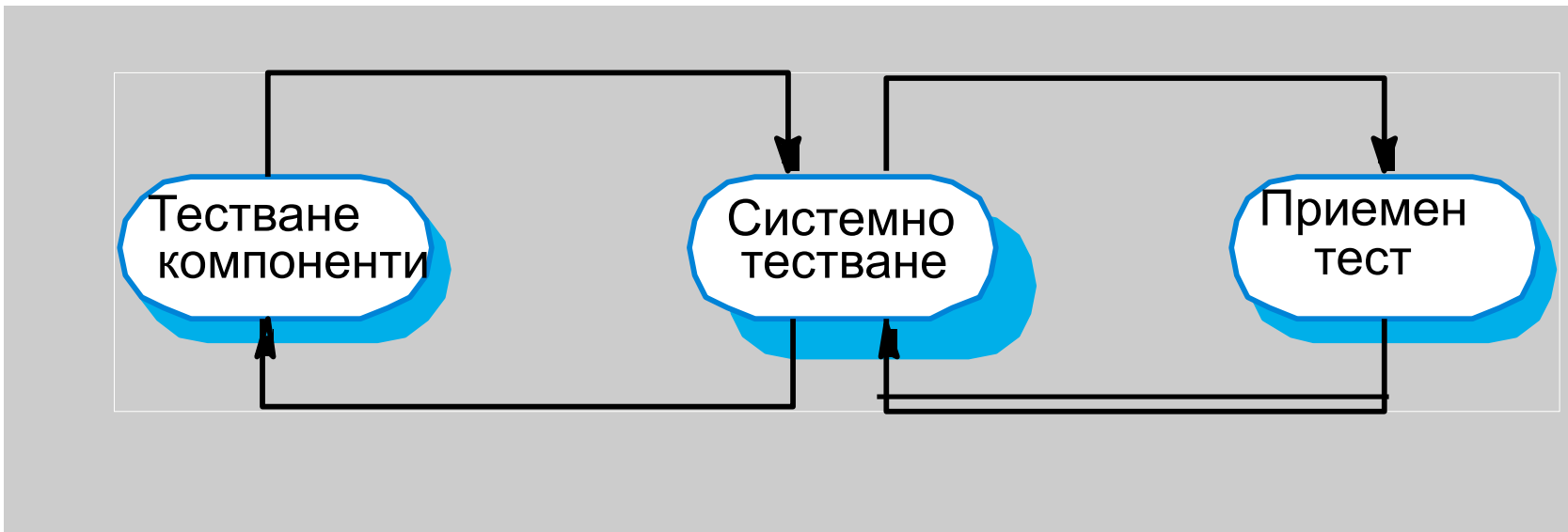
Процесът на настройка



Валидиране на софтуера

- Верификацията и валидирането (V & V) са предназначени да покажат, че системата отговаря на спецификацията си и удовлетворява изискванията на клиента.
- Включва проверка и преглед на процесите и тестване на системата.
- Тестването включва изпълнението на системата с тестови примери, извлечени от спецификацията на реалните данни обработвани от системата.

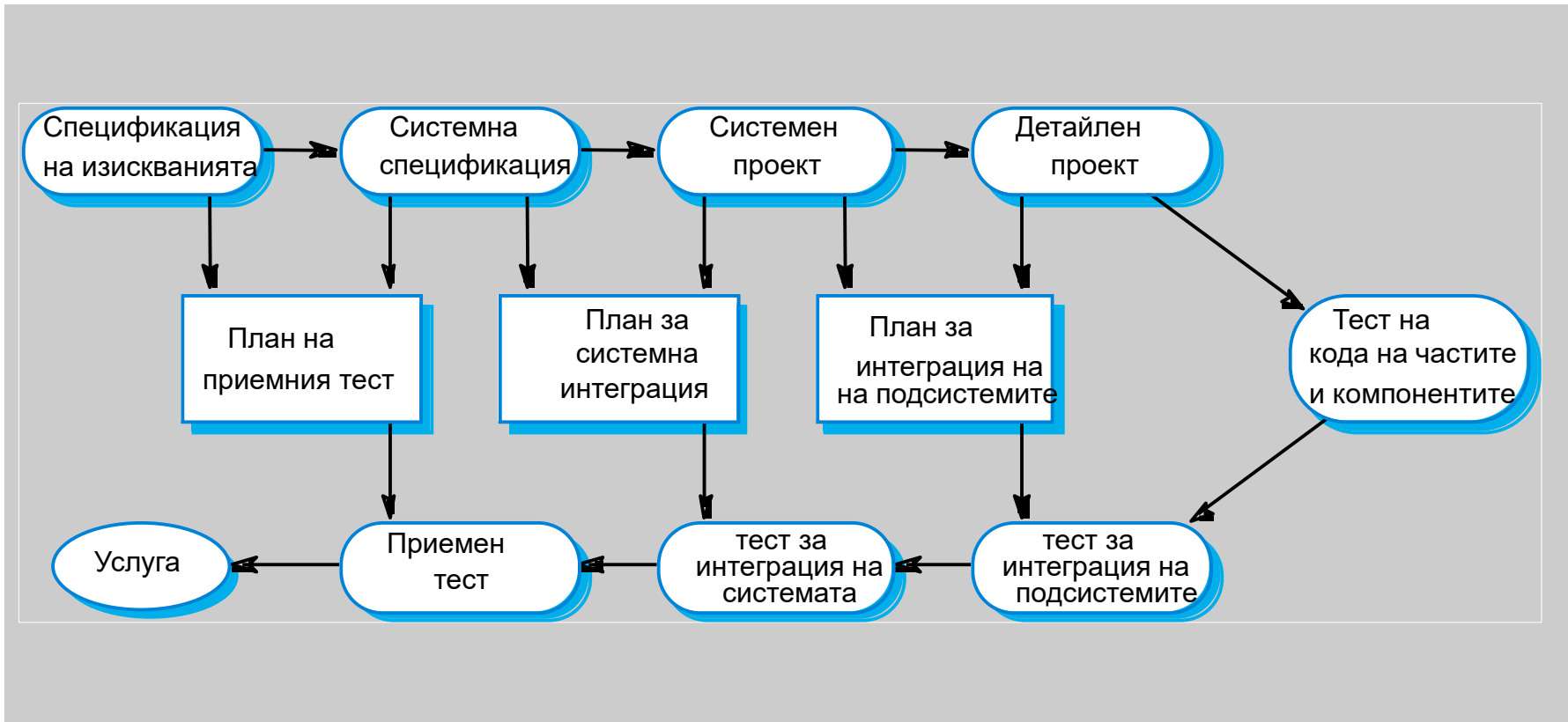
Процес на тестване



Етапи на тестването

- Тестване на компонентите или съставните части
  - Индивидуалните компоненти се тестват независимо
  - Компонентите могат да бъдат функции или обекти или свързани групи от такива единици.
- Системно тестване
  - Системата се тества като едно цяло. Особено важно е тестването на нововъзникналите свойства.
- Приемен тест
  - Тестване с данни на клиента, за да се провери дали системата удовлетворява нуждите на клиента.

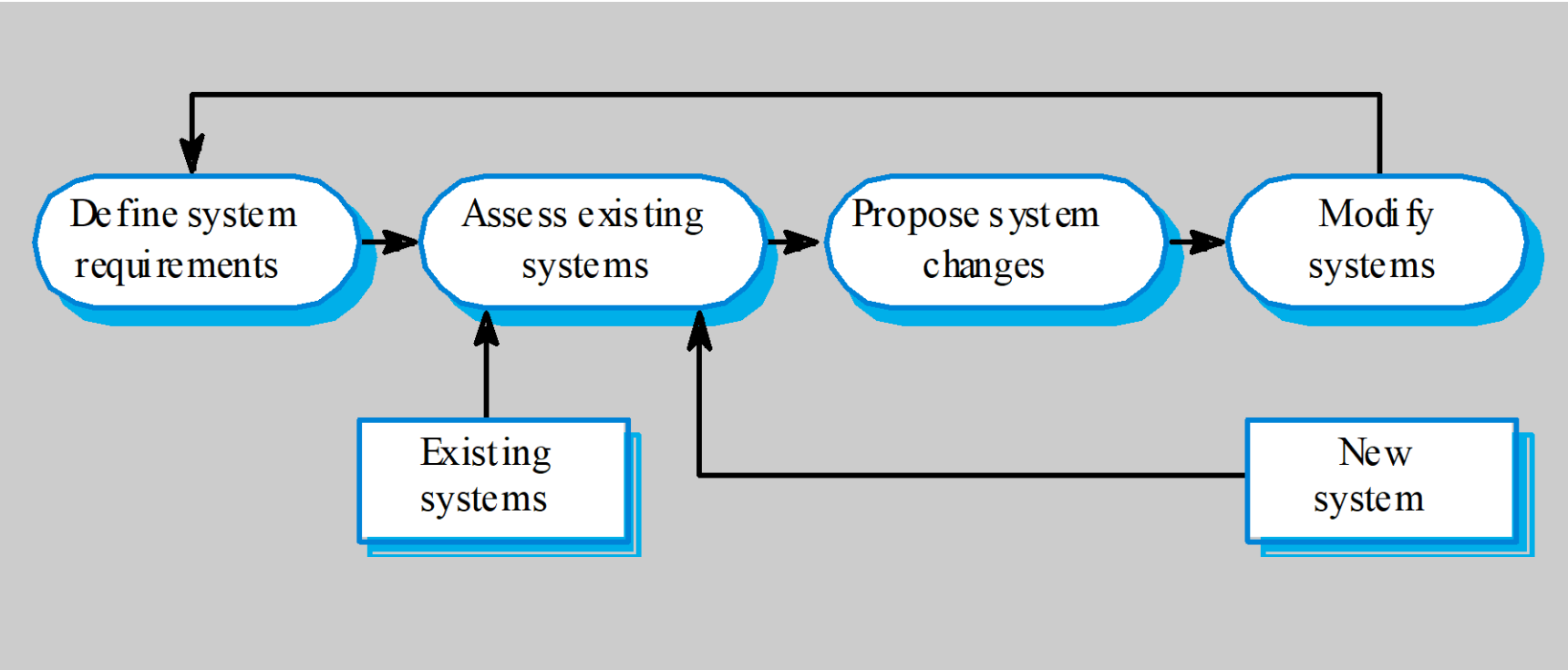
Фази на тестването



Развитие (еволюция) на софтуера

- Софтуерът по природа е гъвкав и може да се променя
- Както изискванията се променят с бизнес обстоятелствата, така и софтуерът поддържащ този бизнес трябва да се развива и променя.
- Въпреки че съществува разделение между разработка и развитие (поддръжка), то е все повече неподходящо, тъй като все по-малко системи са напълно нови.

Развитие на системата



RUP (Рационален унифициран процес)

- Модерен модел на софтуерния процес, произхождаш от работите върху UML
- Обикновено се описва от три гледни точки (перспективи):
  - Динамична, която показва фазите във времето;
  - Статична, която показва дейностите на процеса
  - Практична, която предлага добрите практики, които да се използват в процеса

RUP модел на фазите

RUP фази

- Начало
  - Установява бизнес фактите за системата и какъв ще приносът й.
- Уточнение
  - Разучава областта на проблема и системната архитектура.
- Изграждане
  - Проектиране, програмиране и тестване
- Transition
  - Внедрява системата в работното й обкръжение

Добри практики на RUP

- Софтуерът се разработва итеративно
- Изискванията се управляват
- Използва се компонентно базирана архитектура
- Софтуерът се моделира визуално
- Верифицира се качеството
- Контролират се промените в софтуера

Статични дейности (**Workflows**)

Workflow	**Описание**
Business modelling Бизнес моделиране	Бизнес процесите се моделират като се използват примери за използване (use cases)
Requirements Изисквания	Идентифицират се актьорите, които взаимодействат със системата и се разработват примери за използване, за да се моделират системните изисквания
Analysis and design Анализ и проектиране	Създава се и се документира проектен модел, като се използват архитектурни модели, компонентни модели, обектни модели и модели на последователности.
Implementation Осъществяване	Осъществяват се компонентите и структурират в подсистеми. Процесът може да се ускори с използването на автоматично генериране на код от проектния модел
Test Тест	Тестването е итеративен процес и се изпълнява заедно с осъществяването. Системното тестване следва осъществяването.
Deployment Предаване	Създава се версия на продукта и се инсталира при потребителите
Configuration and change management Конфигуриране и управление на промените	Тази поддържаща дейност управлява промените (вж. Гл.29)
Project management Управление на проекта	Тази поддържаща дейност системната разработка (Гл.5)
Environment Обкръжение	Тази дейност е свързана с направата на подходящи софтуерни инструменти за екипа от разработчици.

Computer-aided software engineering (CASE)

- CASE е софтуер, който подпомага разработката на софтуера и процесите на развитие.
- Автоматизирани дейности
  - Графични редактори за разработка на модела на системата
  - Речник на данните за управление на обектите на проекта
  - Визуален конструктор на потребителски интерфейс
  - Дебъгери за подпомагане на откриването на дефекти
  - Автоматични транслатори за генериране на нови версии на програмата

Case технология

- Case технологията води до чувствително подобряване на софтуерния процес. Обаче не в степента, която беше някога предричана.
  - Софтуерното инженерство изисква креативно мислене, което не може да се автоматизира
  - Софтуерното инженерство е екипна дейност, и при големите проекти се губи много време за взаимодействие между участниците. Това всъщност не се подпомага от Case технологията

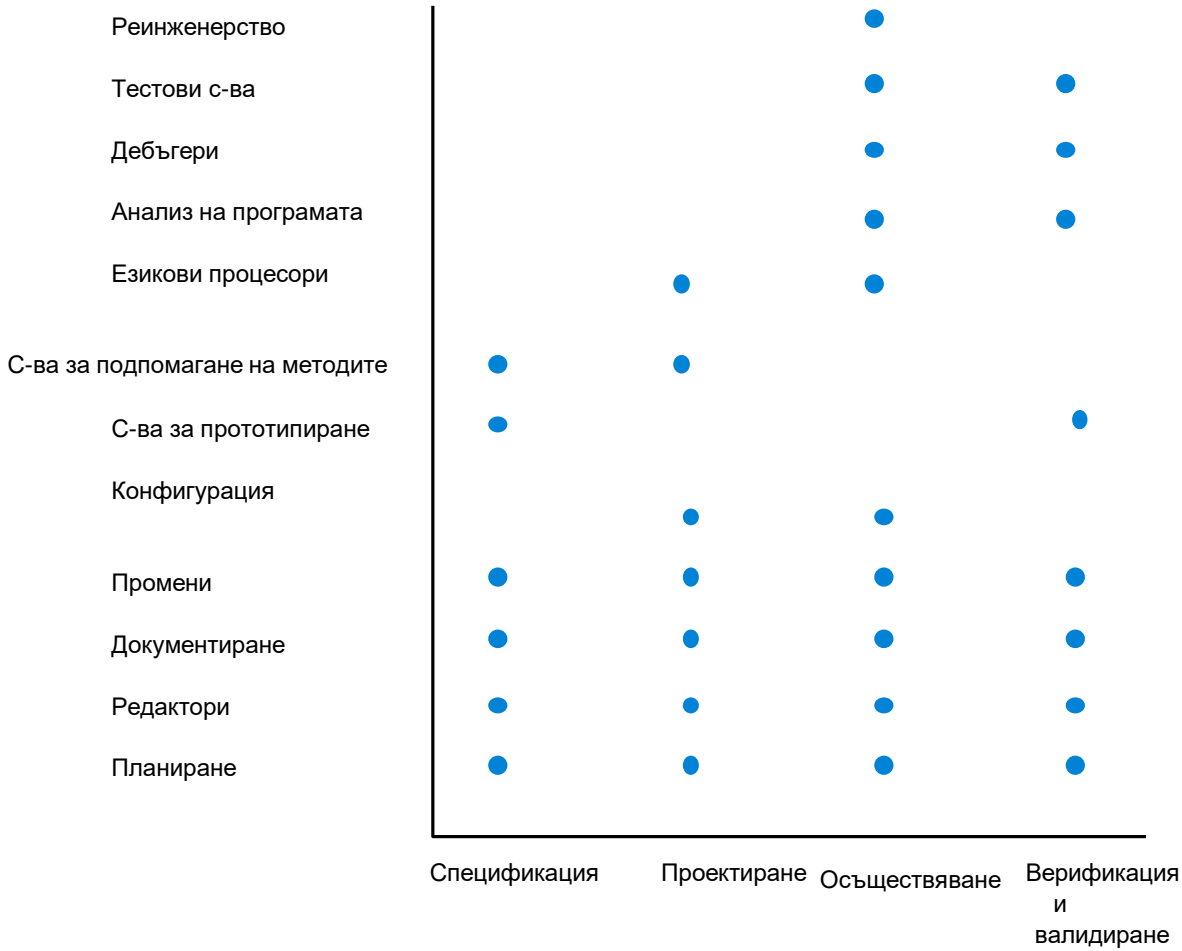
CASE класификация

- Класификацията помага за разбирането на различните типове CASE средства и как те подпомагат дейностите в процеса.
- Функционална перспектива
  - Класификация според специфичните функции
- Перспектива от гледна точка на процеса
  - Класификация според подпомаганите дейности
- Интеграционна перспектива
  - Средствата се класифицират според това как са организирани в интегрираните единици, които подпомагат една или повече дейности.

Функционална класификация

Тип	Примери
Средства за управление	PERT средства, средства за оценка, електронни таблици
Редактиращи средства	Текст редактори, текст процесори, редактори на диаграми
Управление на промените	С-ва за проследяване на изискванията, за контрол на промените
Управление на конфигурацията	С-ва за управление на версиите, за изграждане на ситемите
Средства за прототипиране	Езици от много високо ниво, генератори на потребителски интерфейс
Средства за поддръжка на методи	Редактори на проекти, речници на данни, генератори на код
Езикови процесори	Компилатори, интерпретатори
Средства за анализ на програми	Крос-референс генератори, статични и динамични анализатори
Средства за тестване	Генератори на тестови данни, сравняване на файлове
Средства за дебъгване	Интерактивни дебъгери
Средства за документиране	Програми за страниране, графични редактори
Средства за реенджиниринг	Системи за кросрерферънс и за програмно реструктуриране

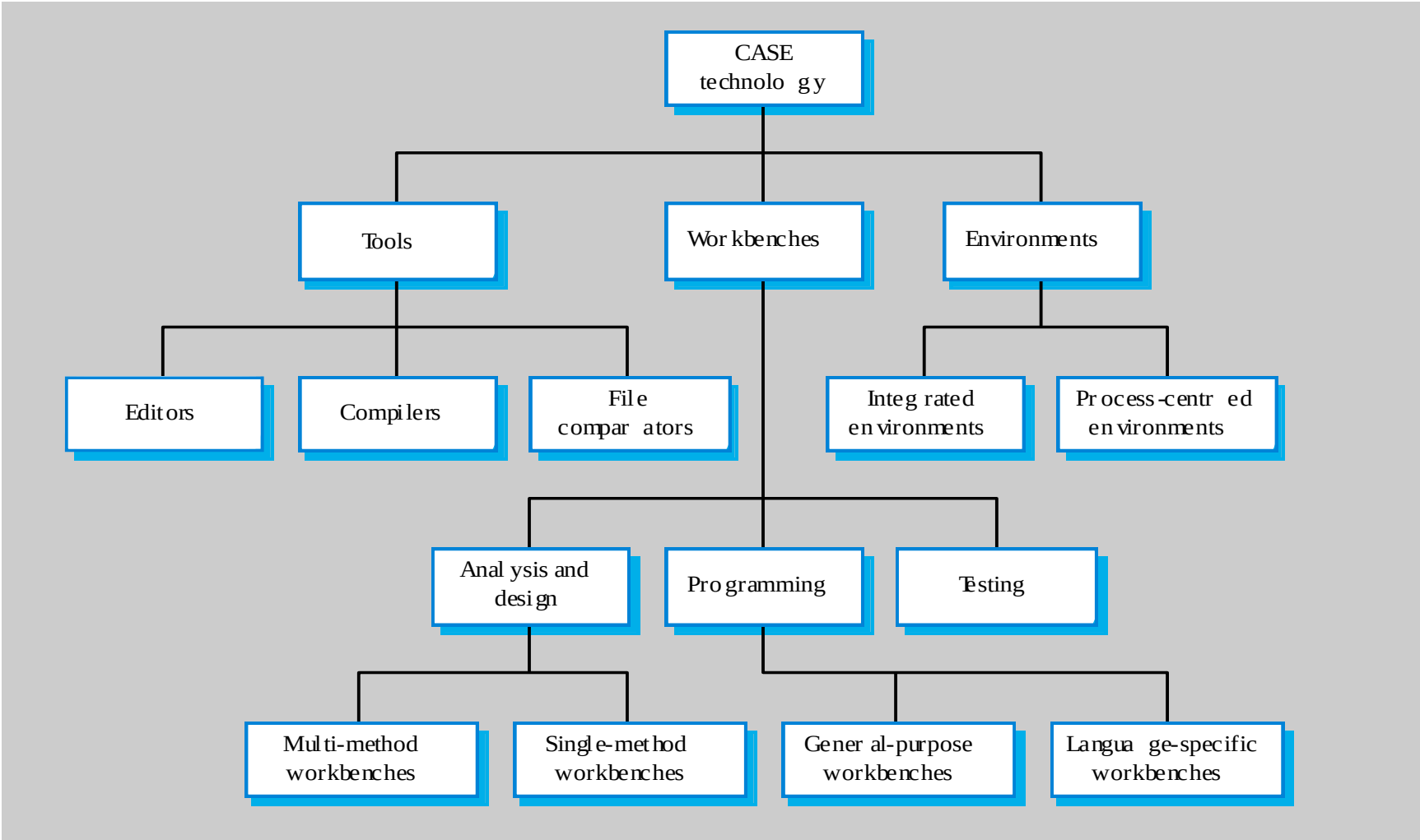
Класификация на средствата по дейности



CASE интегриране

- Средство
  - Подпомагат индивидуални дейности като проверка на съвместимостта, редактиране на текст и др.
- Workbench
  - Подпомага фаза на процеса като спецификация, проектиране. Обикновено включва няколко интегрирани средства
- Интегрирана среда
  - Подпомага целия процес или съществена негова част. Обикновено включва няколко интегрирани Workbench-а.

Средства, workbench среди



Заключение

- Софтуерните процеси са дейностите включени в производството и развитието на софтуерни системи.
- Моделите са абстрактни представяния на софтуерните процеси
- Основни дейности са спецификацията, проектирането, валидирането и развитието.
- Обобщените модели описват организацията на софтуерния процес. Такива са “waterfall”, еволюционната разработка и компонентно базирания инженеринг.
- Итеративните модели описват софтуерния процес като цикъл от дейности.

Заключение

- Инженерингът на изискванията е процес на разработка на спецификацията на софтуера.
- Проектирането и осъществяването превръщат спецификацията в изпълнима програма.
- Валидирането включва проверки дали системата отговаря на спецификацията и на нуждите на потребителя.
- Развитието е свързано с промените в системата след като е пусната в действие.
- RUP (Rational Unified Process) е обобщен модел, който отделя дейностите от фазите
- CASE технологията подпомага дейностите на софтуерния процес

Благодаря за вниманието!

УСП - 3 НН